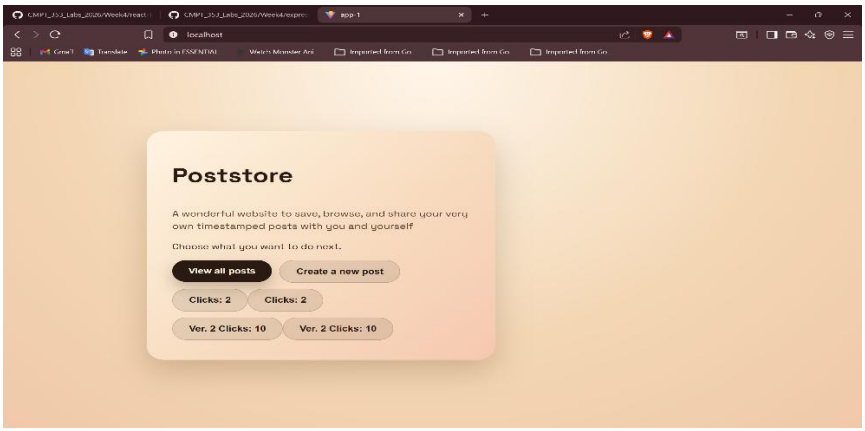
Assignment 04

NAME: Lamisa Aziz || NSID: zlq765 || STUDENT ID: 11380653 || COURSE: CMPT_353

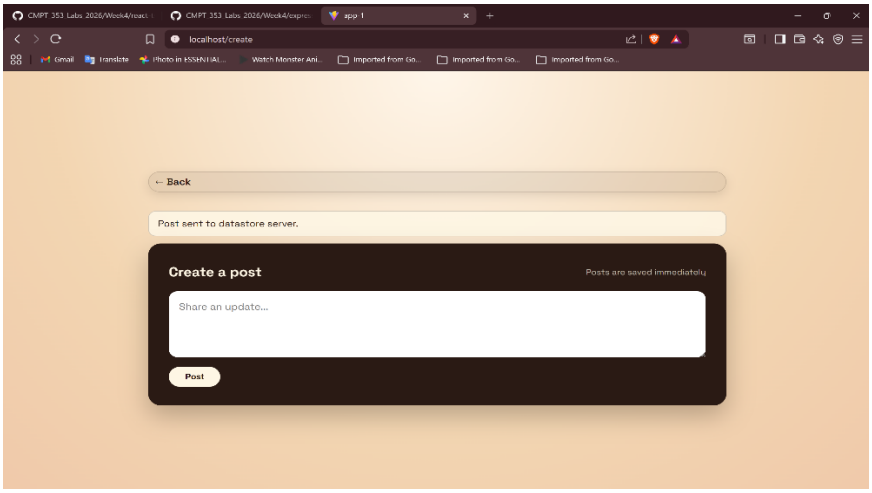
Part C: Test Report

1) Screenshots:

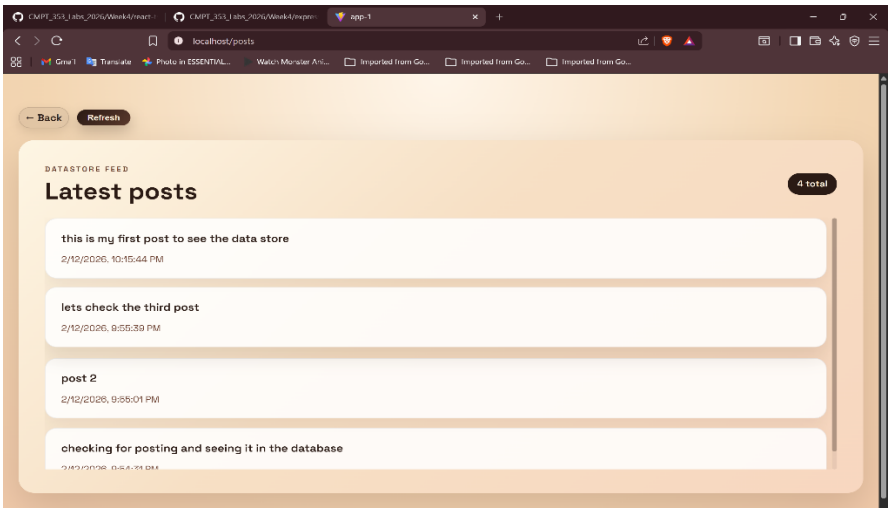
● Landing page

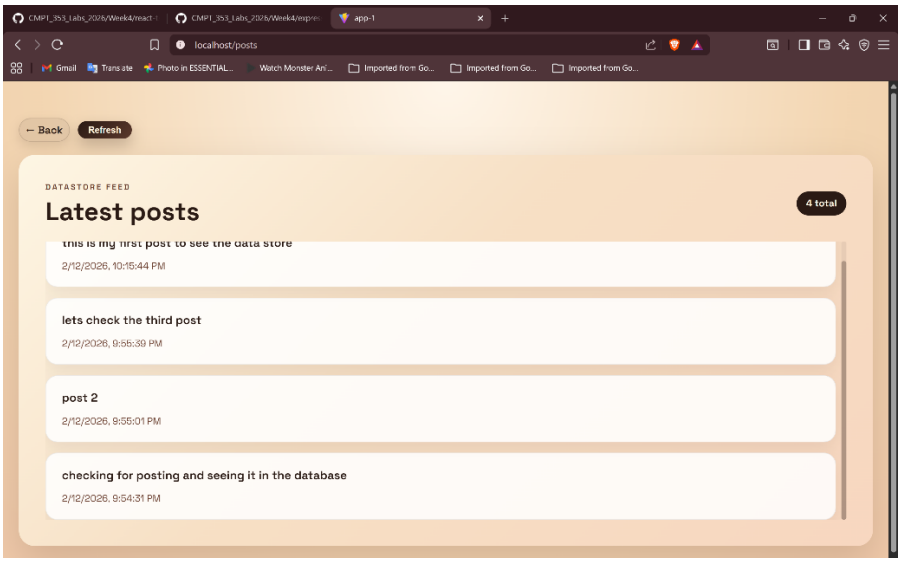


● Create Posts page



● Post page





2) Overall Testing Approach:

● Backend testing (endpoints, JSON format, validation, error cases)

I tested the backend using:

- Browser fetch calls (via the frontend)
- Manual curl commands
- Observing backend logs in Docker
- Checking MySQL persistence across restarts

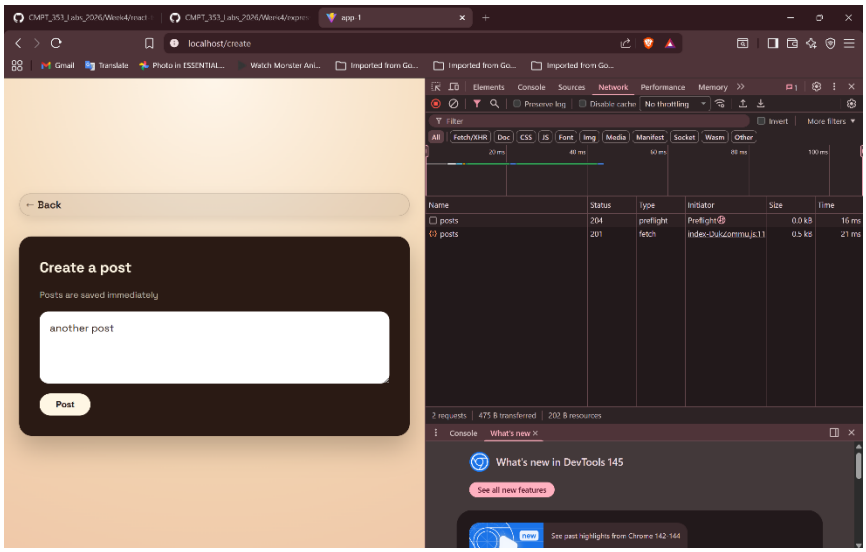
I verified:

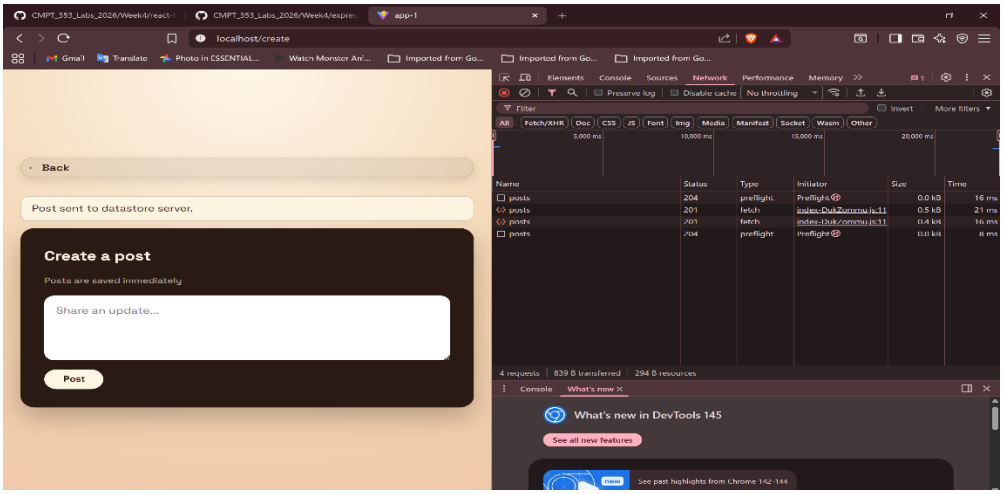
- **GET /api/posts** returns a JSON array with id, topic, data, and timestamp
- **POST /api/posts** accepts valid JSON and returns 201 Created
- Validation errors return 400 Bad Request
- Posts are stored in MySQL and persist after container restarts
- Ordering is consistent (newest first)

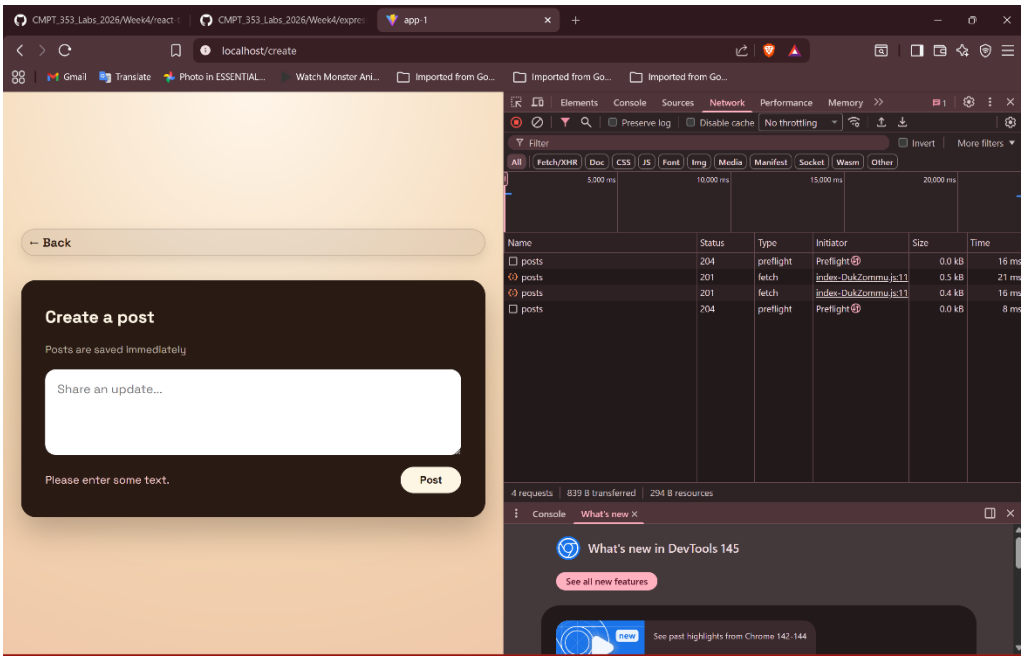
● **Frontend testing (navigation, fetch calls, creating & display flow)**

I tested the frontend by:

- Navigating between /, /posts, and /create using react-router
- Creating posts and verifying they appear immediately
- Checking that validation errors appear for empty input
- Confirming fetch calls succeed (status 201/200)
- Using Chrome DevTools → Network tab to inspect requests
- Refreshing /posts to confirm data loads from backend




localhost/create
Back
Post sent to datastore server.
Create a post
Posts are saved immediately
Share an update...
Post
Elements
Console
Sources
Network
Performance
Memory
Name
Status
Type
Initiator
Size
Time
posts 204 preflight Preflight 0.0 kB 16 ms
posts 201 fetch index-DukZommu.js:11 0.5 kB 21 ms
posts 201 fetch index-DukZommu.js:11 0.4 kB 16 ms
posts 204 preflight Preflight 0.0 kB 8 ms
4 requests | 839 B transferred | 294 B resources
What's new in DevTools 145
See all new features


localhost/create
← Back
Create a post
Posts are saved immediately
Share an update...
Please enter some text.
Post
Elements
Console
Sources
Network
Performance
Memory
Name
Status
Type
Initiator
Size
Time
posts 204 preflight Preflight 0.0 kB 16 ms
posts 201 fetch index-DukZommu.js:11 0.5 kB 21 ms
posts 201 fetch index-DukZommu.js:11 0.4 kB 16 ms
posts 204 preflight Preflight 0.0 kB 8 ms
4 requests | 839 B transferred | 294 B resources
What's new in DevTools 145
See all new features

3) Functional Test Cases

Test Case	Input	Expected Result	Actual Result
Valid post creation	topic="Hello", data="World"201	201 Created, post appears in list with timestamp	Passed - post stored and displayed
Missing topic	topic="", data="Test"	400 Bad Request with JSON error	Passed - frontend shows “Failed to fetch” and backend logs validation error
Backend down	Stop backend container	Frontend shows “Failed to fetch”	Passed - error message displayed

4) Load Testing (using loadtest)

- The exact command I used:

npx loadtest --concurrency=10 --maxRequests=1000 http://localhost:8080/api/posts

```
Target URL:        http://localhost:8080/api/posts
Max requests:      1000
Concurrent clients: 40
Running on cores:   4
Agent:             none

Completed requests: 1000
Total errors:       0
Total time:         2.945 s
Mean latency:       109.1 ms
Effective rps:      340

Percentage of requests served within a certain time
  50%     110 ms
  90%     118 ms
  95%     122 ms
  99%     134 ms
 100%     178 ms (longest request)
```

Load Test Analysis:

I ran a load test using 1000 requests with 40 concurrent clients against the GET /api/posts endpoint. The server successfully handled all 1000 requests with **0 errors**, completing the test in **2.945 seconds**. The mean latency was around **109 ms**, and the effective throughput was **340 requests per second**, indicating that the Express + MySQL backend performs reliably under moderate load. These results show that the system remains responsive and stable even with concurrent traffic.

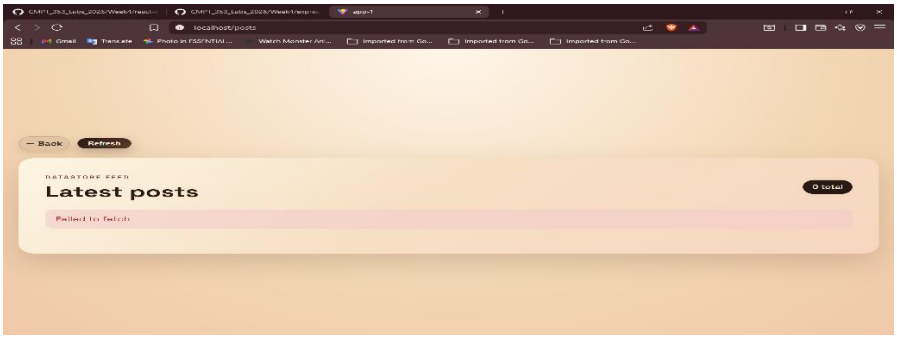
5) Challenges and Solutions Briefly list issues you encountered and how you solved them.

Docker networking issues - Initially, the frontend could not reach the backend because the fetch URL used http://backend:8080. The fix was to use http://localhost:8080 since the browser runs outside Docker.

Database readiness race condition - The backend started before MySQL was ready. Adding a waitForDb() loop solved the issue.

Frontend not displaying post text - The backend returned data, but the frontend expected text. Updating Posts.tsx to use post.data fixed the display.

Validation errors Missing topic/data caused “Failed to fetch”. This was resolved by proper backend validation and frontend error handling.



Folder structure compliance Ensured /dist was generated by npm run build and included in the final ZIP.